

这次就不太方便用倒酱油做比喻了：硬着头皮把醋倒在酱油瓶子里，然后分离出酱油倒回醋瓶子？比较理性的方法是手工模拟这段程序，看看每条语句执行后的情况。

在顺序结构程序中，程序一条一条依次执行。为了避免值和变量名混淆，假定用户输入的是 a_0 和 b_0 ，因此 `scanf` 语句执行完后 $a = a_0$ ， $b = b_0$ 。

执行完 $a = a + b$ 后： $a = a_0 + b_0$ ， $b = b_0$ 。

执行完 $b = a - b$ 后： $a = a_0 + b_0$ ， $b = a_0$ 。

执行完 $a = a - b$ 后： $a = b_0$ ， $b = a_0$ 。

这样，就不难理解两个变量是如何交换的了。

提示 1-12：可以通过手工模拟的方法理解程序的执行方式，重点在于记录每条语句执行之后各个变量的值。

这个方法看起来很好（少用一个变量），但实际上很少使用，因为它的适用范围很窄：只有定义了加减法的数据类型才能采用此方法^①。事实上，笔者并不推荐读者采用这样的技巧实现变量交换：三变量法已经足够好，这个例子只是帮助读者提高程序阅读能力。

提示 1-13：交换两个变量的三变量法适用范围广，推荐使用。

那么是不是说，三变量法是解决本题的最佳途径呢？答案是否定的。多数算法竞赛采用黑盒测试，即只考查程序解决问题的能力，而不关心采用了什么方法。对于本题而言，最合适的程序如下：

程序 1-10 变量交换（3）

```
#include<stdio.h>
int main()
{
    int a, b;
    scanf("%d%d", &a, &b);
    printf("%d %d\n", b, a);
    return 0;
}
```

换句话说，我们的目标是解决问题，而不是为了写程序而写程序，同时应保持简单（Keep It Simple and Stupid, KISS），而不是自己创造条件去展示编程技巧。

提示 1-14：算法竞赛是在比谁能更好地解决问题，而不是在比谁写的程序看上去更高级。

1.4 分支结构程序设计

例题 1-4 鸡兔同笼

已知鸡和兔的总数量为 n ，总腿数为 m 。输入 n 和 m ，依次输出鸡的数目和兔的数目。

^① 这个方法还有一个“变种”：用异或运算“^”代替加法和减法，还可以进一步简写成 $a^=b^=a^=b$ ，但不建议使用。

如果无解，则输出 No answer。

样例输入：

14 32

样例输出：

12 2

样例输入：

10 16

样例输出：

No answer

【分析】

设鸡有 a 只，兔有 b 只，则 $a+b=n$ ， $2a+4b=m$ ，联立解得 $a=(4n-m)/2$ ， $b=n-a$ 。在什么情况下此解“不算数”呢？首先， a 和 b 都是整数；其次， a 和 b 必须是非负的。可以通过下面的程序判断：

程序 1-11 鸡兔同笼

```
#include<stdio.h>
int main()
{
    int a, b, n, m;
    scanf("%d%d", &n, &m);
    a = (4*n-m)/2;
    b = n-a;
    if(m % 2 == 1 || a < 0 || b < 0)
        printf("No answer\n");
    else
        printf("%d %d\n", a, b);
    return 0;
}
```

上面的程序用到了 if 语句，其一般格式是：

```
if(条件)
    语句 1;
else
    语句 2;
```

注意语句 1 和语句 2 后面的分号，以及 if 后面的括号。“条件”是一个表达式，当该表达式的值为“真”时执行语句 1，否则执行语句 2。另外，“else 语句 2”是可以省略的。

语句 1 和语句 2 前面的空行是为了让程序更加美观，并不是必需的，但强烈推荐读者使用。

提示 1-15: if 语句的基本格式为：if(条件) 语句 1; else 语句 2。

换句话说，“ $m\%2==1 \parallel a<0 \parallel b<0$ ”是一个表达式，其字面意思是“m 是奇数，或者 a 小于 0，或者 b 小于 0”。这句话可能正确，也可能错误。因此这个表达式的值可能为真，也可能为假，取决于 m、a 和 b 的具体数值。

这样的表达式称为逻辑表达式。和算术表达式类似，逻辑表达式也由运算符和值构成，例如“ $\parallel$ ”运算符称为“逻辑或”， $a \parallel b$ 表示 a 为真，或者 b 为真。换句话说，a 和 b 只要有一个为真， $a \parallel b$ 就为真；如果 a 和 b 都为真，则 $a \parallel b$ 也为真。和其他语言不同的是，在 C 语言中单个整数也可以表示真假，其中 0 为假，其他值为真。

提示 1-16: if 语句的条件是一个逻辑表达式，它的值可能为真，也可能为假。单个整数值也可以表示真假，其中 0 为假，其他值为真。

细心的读者也许发现了，如果 a 为真，则无论 b 的值如何， $a \parallel b$ 均为真。换句话说，一旦发现 a 为真，就不必计算 b 的值。C 语言正是采取了这样的策略，称为短路(short-circuit)。也许读者会觉得，用短路的方法计算逻辑表达式的唯一优点是速度更快，但其实并不是这样，稍后将通过几个例子予以证实。

提示 1-17: C 语言中的逻辑运算符都是短路运算符。一旦能够确定整个表达式的值，就不再继续计算。

例题 1-5 三整数排序

输入 3 个整数，从小到大排序后输出。

样例输入：

20 7 33

样例输出：

7 20 33

【分析】

a、b、c 这 3 个数一共只有 6 种可能的顺序：*abc*、*acb*、*bac*、*bca*、*cab*、*cba*，所以最简单的思路是使用 6 条 if 语句。

程序 1-12 三整数排序 (1) (错误)

```
#include<stdio.h>
int main()
{
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    if(a < b && b < c)printf("%d %d %d\n", a, b, c);
    if(a < c && c < b)printf("%d %d %d\n", a, c, b);
```

```

    if(b < a && a < c)printf("%d %d %d\n", b, a, c);
    if(b < c && c < a)printf("%d %d %d\n", b, c, a);
    if(c < a && a < b)printf("%d %d %d\n", c, a, b);
    if(c < b && b < a)printf("%d %d %d\n", c, b, a);
    return 0;
}

```

上述程序看上去没有错误，而且能通过题目中给出的样例，但可惜有缺陷：输入“111”将得不到任何输出！这个例子说明：即使通过了题目中给出的样例，程序仍然可能存在问题。

提示 1-18： 算法竞赛的目标是编程对任意输入均得到正确的结果，而不仅是样例数据。

将程序稍作修改：把所有的小于符号“<”改成小于等于符号“<=”（在一个小于号后添加一个等号）。这下总可以了吧？很遗憾，还是不行。对于“111”，6种情况全部符合，程序一共输出了6次“111”。

一种解决方案是人为地让6种情况没有交叉：把所有的if改成else if。

程序 1-13 三整数排序（2）

```

#include<stdio.h>
int main()
{
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    if(a <= b && b <= c) printf("%d %d %d\n", a, b, c);
    else if(a <= c && c <= b) printf("%d %d %d\n", a, c, b);
    else if(b <= a && a <= c) printf("%d %d %d\n", b, a, c);
    else if(b <= c && c <= a) printf("%d %d %d\n", b, c, a);
    else if(c <= a && a <= b) printf("%d %d %d\n", c, a, b);
    else if(c <= b && b <= a) printf("%d %d %d\n", c, b, a);
    return 0;
}

```

最后一条语句还可以简化成单独的else（想一想，为什么），不过，幸好程序正确了。

提示 1-19： 如果有多个并列、情况不交叉的条件需要一一处理，可以用else if语句。

另一种思路是把 a 、 b 、 c 这3个变量本身改成 $a \leq b \leq c$ 的形式。首先检查 a 和 b 的值，如果 $a > b$ ，则交换 a 和 b （利用前面讲过的三变量交换法）；接下来检查 a 和 c ，最后检查 b 和 c ，程序如下：

程序 1-14 三整数排序（3）

```

#include<stdio.h>
int main()
{

```